

O artigo inicia ressaltando que, na era digital, praticamente todos os setores dependem de softwares que armazenam informações críticas e realizam transações vitais. Essa onipresença atrai ataques cada vez mais sofisticados, tornando imperativo incorporar ~~monitorear~~ segurança desde a concepção até a manutenção do software. Conceitos fundamentais como confidencialidade, Integridade e não-repúdio devem orientar todos os fases do ciclo de vida do desenvolvimento seguro de software (SSDLC).

Para estruturar essa proteção, o texto apresenta práticas essenciais baseadas em princípios como "defesa em profundidade", modelo positivo de segurança, falta segura e privacidade mínima. Destaca-se também a importância de evitar "segurança por obscuridade", manter a simplicidade das soluções, detectar intrusões por meio de logs e monitoramento, e não confiar cegamente em infraestruturas ou serviços externos.

Na fase de engenharia de requisitos de segurança, recomenda-se o uso de técnicas como análise SMART, casos de mal uso e casos de abuso e metodologias como STORRE, SQUARE e MSRE para modelar e priorizar ameaças. Durante a arquitetura e o design, a modelagem de ameaças e padrões de design seguro ajudam a antecipar ataques e fortalecer pontos de confiança. Na implementação, enfatiza-se "secure by design", "secure by default" e codificação defensiva, apoiadas por frameworks como ESAPI e pela atenção à complexidade de código. Já nos testes de segurança, devem ser combinados tests de unidade e integração, varredura de vulnerabilidades, fuzzing e Pen-testing para validar controles e identificar brechas.